Computation for Analytics

DIANE WOODBRIDGE, PH.D

Content

How to approach programming

Programming Patterns

- Map
- Accumulate
- Combine
- Split
- Slice
- Filter/Search
- Matrix Processing

More Examples

Content

How to approach programming

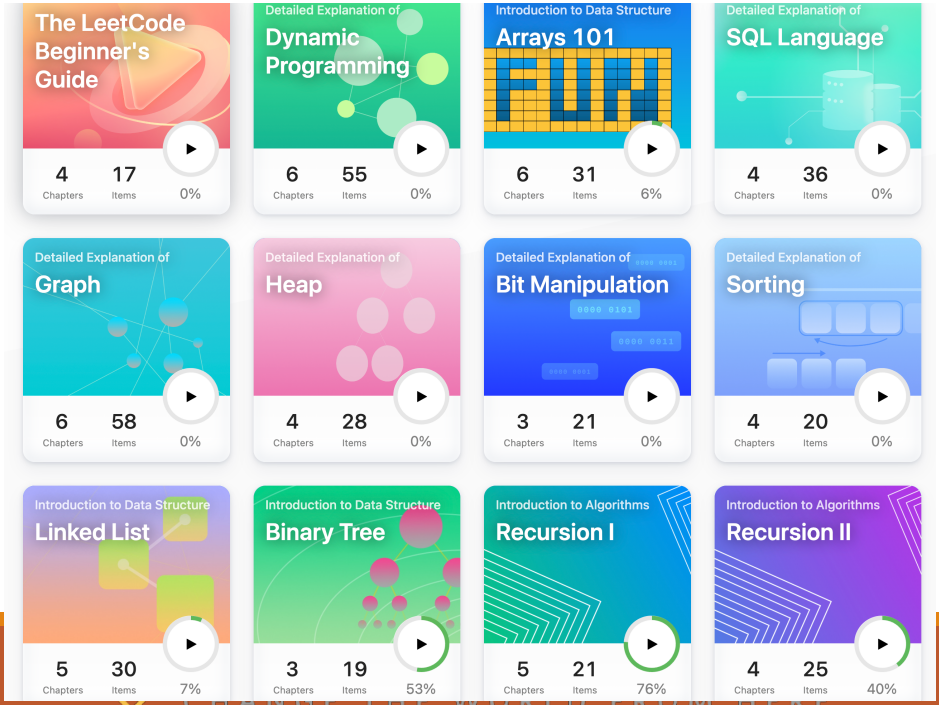
Programming Patterns

- Accumulate
- Map
- Combine
- Split
- Slice
- Filter/Search
- Matrix Processing

More Examples

Problem Solving through Programs

- Experienced programmers start from a collection of generic high-level mental templates to classify the problem into a few categories.
- Gaining experience as a programmer means recognizing and remembering patterns in your code, both high and low level



https://leetcode.com/explore/learn/

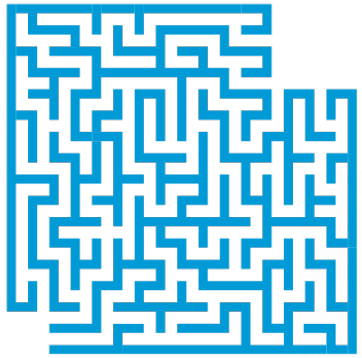
Example: Data Science ETL Pipeline

- **Acquire data**, which means finding a suitable file or collecting data from the web and storing it in a file or database
- **Load data** from disk or database and organize into a suitable data structure in memory
- **Pre-process data** including normalizing, filtering, and cleaning data
- **Process the data**, which can mean training a machine learning model, transforming the data, computing summary statistics, or optimizing a cost function
- **Emit results**, which can be anything from simply printing, storing data or model to the disk to generating a fancy visualization

Basic problem-solving strategy



Communicate - Make sure to understand requirement, assumption, expected input and output, test cases and edge-cases



Start with the end result and work your way backwards

- You might want to use a scratch paper to design your algorithm
- A prior step of 'step *i*' should compute the variable required by 'step *i*'
  - E.g., median: to pick middle value, previous step must sort data



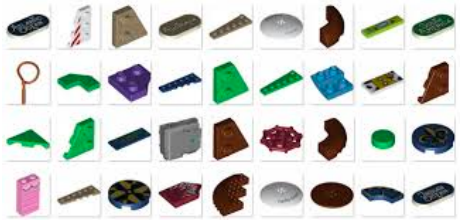
High-level vs. Low-level Programming Patterns

High-level templates help us organize the thoughts and plan our program



Low-level patterns are Lego pieces that we put together to fill in details.

- This low-level patterns could be very different by each individual - and that's why I asked to work on a high level to avoid plagiarism.



Sample programming patterns

You're no doubt familiar with simple patterns such as:

- Sum the numbers in a list
- Count the elements in a list

But there are many many more, such as:

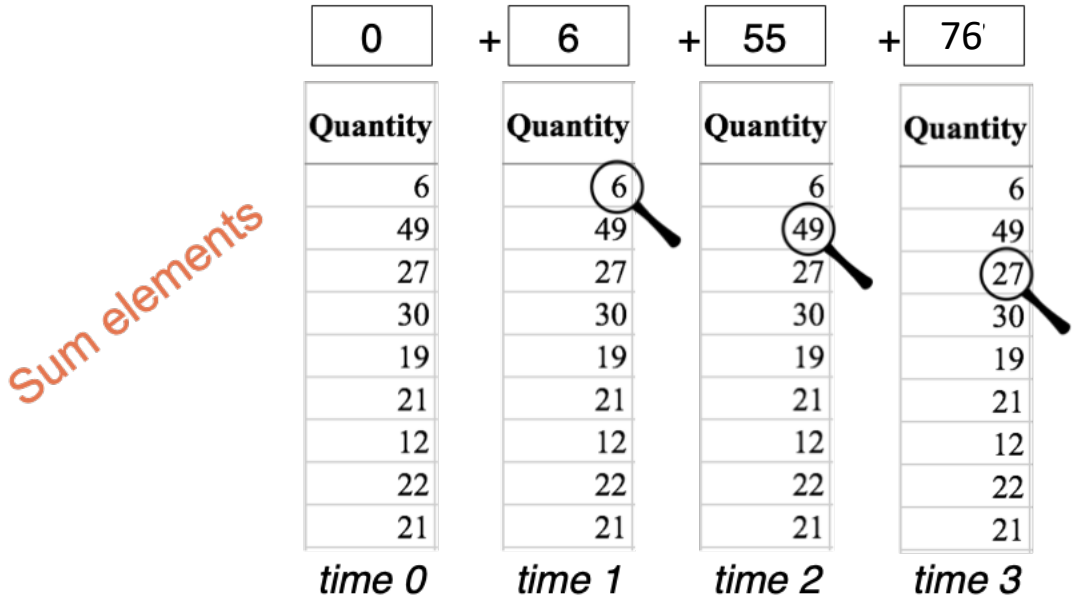
- Find all values in a list satisfying a condition
- Apply an operation to each element of a list to get new list
- Split a list of strings into 2 or more lists
- Merge two sorted lists
- Delete records in a dataframe that satisfy a condition

Think and plan at this level or higher, not the code level

Visualize In/Output ➡ Identify Pattern

Think visually about how you would manipulate lists of data or extract data

- Identify patterns by drawing input/output using small sample data on paper to understand the expectations and come up with your own plan



Matrix addition

1	3	10	+	7	4	0	=	
4	-9	0		4	3	1		-6
2	5	8		1	6	8		

Content

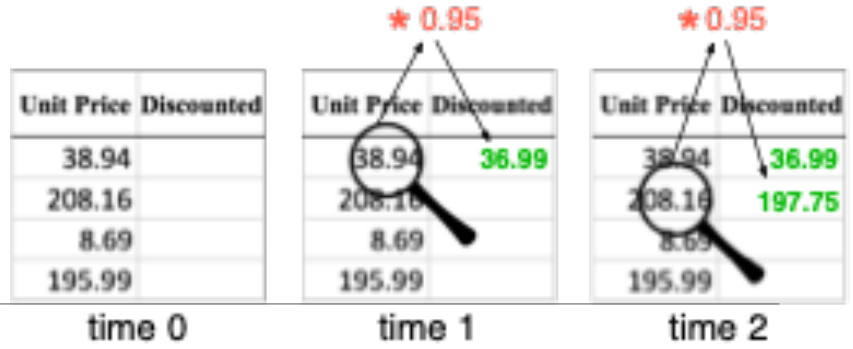
How to approach programming

Programming Patterns

- Map
- Accumulate
- Combine
- Split
- Slice
- Filter/Search
- Matrix Processing

More Examples

Programming Patterns



Map

- Map/Convert one sequence to another, applying an operator or function to each element
  - Ex. A discounted unit price.

```
unit_price = [38.94, 208.16, 8.69, 195.99]
discounted = list() # empty list
for price in unit_price:
    discounted.append(round(price * 0.95, 2))

# list comprehension shines here!!
discounted = [round(p*0.95, 2) for p in unit_price]
```

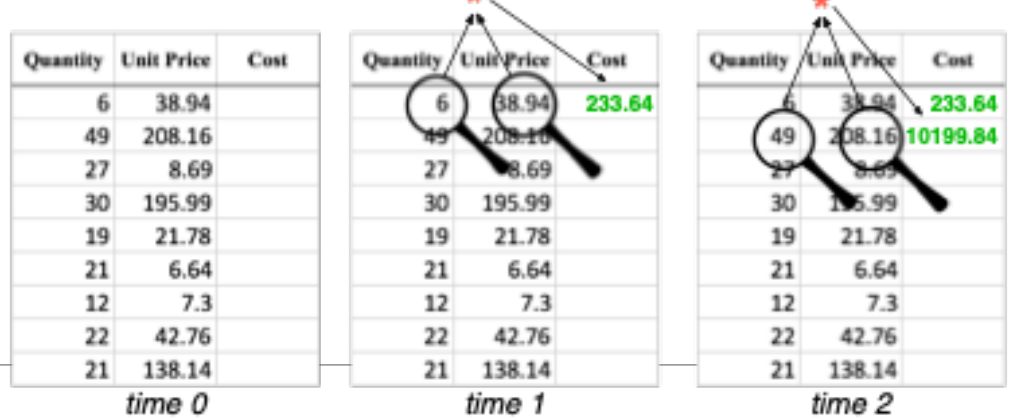
Programming Patterns

Accumulate

- Traverse a sequence of elements and accumulate a value
  - Can use any other arithmetic operator, such as *
- Called reduce, as in MapReduce in distributed computing

```
quantity = [6, 49, 27, 30, 19, 21, 12, 22, 21]
sum = 0
for q in quantity:
    sum += q            # same as: sum = sum + q
print(sum)
```

Programming Patterns



Combine

- Traverse two or more lists at once placing the result in a third list
  - Ex. Multiply the i-th element from different sequences and placing the result in an output sequence.

```
quantity = [6, 49, 27, 30]
unit_price = [38.94, 208.16, 8.69, 195.99]

cost = []
for i in range(len(quantity)):
    cost.append( round(quantity[i] * unit_price[i], 2) )

# list comprehension
cost = [round(quantity[i] * unit_price[i], 2) for i in range(len(quantity))]
```

Programming Patterns

Split

- The opposite of combining is splitting where we split a string into two or more strings
  - Ex: split list of full names into their first and last names

```
names = ['Shan Wang', 'Diane Woodbridge', 'Yannet Interian']
first = []
last = []
for name in names:
    f,l = name.split()
    first.append(f)
    last.append(l)
```

E	F
Customer Name	
Muhammed MacIntyre	
Barry French	
Barry French	
Clay Rozendal	
Carlos Soltero	
Carlos Soltero	

E	F
First Name	**Last Name**
Muhammed	MacIntyre
Barry	French
Barry	French
Clay	Rozendal
Carlos	Soltero
Carlos	Soltero

Programming Patterns

Slice

- Extract a subset of a list (that fits in memory) and manipulate
- Syntax is sequence[begin:end]

```
names=['Xue', 'Mary', 'Bob']
print(names[0:1])
print(names[0:2])
print(names[0:3])
print(names[2:3])
print(names[1:])
print(names[-1])
print(names[-2:])
```



```
['Xue']
['Xue', 'Mary']
['Xue', 'Mary', 'Bob']
['Bob’]
['Mary', 'Bob']
Bob
['Mary', 'Bob']
```

Programming Patterns

Filter

- Filter each element based on a given condition.
  - If true, adds the element to the new sequence or remove from the original mutable sequence.

```
shipping = [35, 68.02, 2.99, 3.99, 5.94, 4.95, 7.72, 6.22]
affordable_shipping = []
for x in shipping:
    if x < 10:
        affordable_shipping.append(x)

# list comprehension
affordable_shipping = [x for x in shipping if x < 10] # much easier
```

Programming Patterns

Search

- Search returns the first (or last) position in the sequence that satisfy a specific condition.

```
first=['Xue', 'Mary', 'Robert']      # our given input
target = 'Mary'                      # searching for Mary
index = -1
for i in range(len(first)):          # i is in range [0..n-1] or [0..n)
    if first[i]==target:
        index = i
        break
print(index)
```

Programming Patterns

1	3	10
4	-9	0
2	5	8

Matrix Processing

- Process each cell in a multi-dimensional structure such as an image or matrix using a nested loop

```
nrows = 3
ncols = 3
# column j value varies more quickly than the row i value
for i in range(nrows):
    for j in range(ncols):
        print( i, j )

# row i value varies more quickly than j
for j in range(ncols):
    for i in range(nrows):
        print( i, j )
```

Content

How to approach programming

Programming Patterns

- Accumulate
- Map
- Combine
- Split
- Slice
- Filter/Search
- Matrix Processing

More Examples

Example 1

Complete a function called find_median_array_length() which takes two lists including string values (array1 and array2) and returns the median length of strings included in array1 and array2.

- Ex.

```
array1 = ['Ann', 'Su', 'T']
array2 = ['Diane', 'Jack', 'Jon']
find_median_array_length(array1, array2) == 3
```

https://leetcode.com/problems/median-of-two-sorted-arrays

Example 2

Create remove_element() which removes all occurrences of val in nums in-place, given an integer array nums and an integer val.

Then return the number of elements in nums which are not equal to val.

- Note : The order of the elements in nums may be changed.
- Ex.

```
nums = [3,2,2,3]
val = 3
remove_element(nums, val) == 2
```

https://leetcode.com/problems/remove-element

Example 3

Given a string s and an integer k, reverse the first k characters for every 2k characters counting from the start of the string.

- If there are fewer than k characters left, reverse all of them.
- If there are less than 2k but greater than or equal to k characters, then reverse the first k characters and leave the other as original.
- Ex.

```
s = "abcdefg"
k = 2
reverse_str(s,k) == 'bacdfeg'

s = "abcd"
k = 2
reverse_str(s,k) == 'bacd'
```

https://leetcode.com/problems/reverse-string-ii

Example 4

Given a string s consisting of words and spaces, return the length of the last word in the string.

- Ex.

```
s = "Hello World"
length_of_last_word(s) == 5
```

https://leetcode.com/problems/length-of-last-word

Example 5

Given a 2D integer array matrix, return the transpose of matrix.

- The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.
- Ex.

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]
transpose(matrix) == [[1,4,7],[2,5,8],[3,6,9]]
```

https://leetcode.com/problems/transpose-matrix/

Example 6 - Roman Integers

Roman numerals are represented by seven different symbols: I (1), V (5), X (10) , L (50), C (100), D (500) and M (1000).

- For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. **However**, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

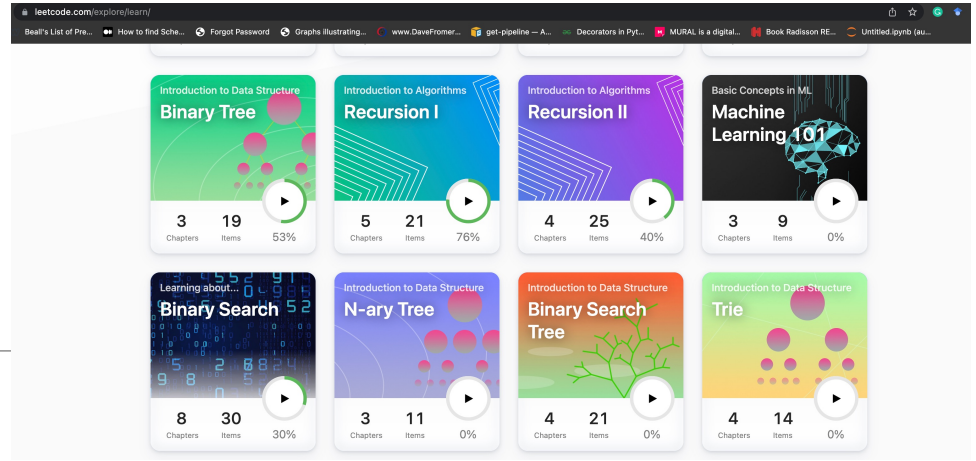
- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Create a function to convert it to an integer, given a roman numeral.
Ex.

```
roman_numbers(s) == 1994
```

https://leetcode.com/problems/roman-to-integer/

More Practice



Get used to some data structures and algorithms.

- We will cover this in the Spring, but a 7 week is not definitely not enough.
- You will encounter coding challenges in your job interviews.
  - Data Scientist - Easy - Medium Difficulty Levels
  - Data Engineer, Machine Learning Engineer - Easy - Hard Difficulty Levels
- Make sure to start with Learn Sections at LeetCode instead of Problems.
  - You will find problem patterns and which data structures and algorithms are efficiently applicable.

Content

How to approach programming

Programming Patterns

- Accumulate
- Map
- Combine
- Split
- Slice
- Filter/Search
- Matrix Processing

More Examples

References

Python : https://docs.python.org/3/

Terence Parr's Note : https://github.com/parrt/msds501/blob/master/notes/python-patterns.ipynb